Objektno orijentirano programiranje
Vježba 3

1. Napišite funkcije
   - `void input_vector(std::vector<int>& v)` koja unosi cijele brojeve dok se ne unese `0` (nula se ne sprema),
   - `void print_vector(const std::vector<int>& v)` koja ispisuje elemente vektora.

   Unesite i ispišite vektor, a zatim ga ispišite obrnutim redoslijedom (`rbegin()`, `rend()`).
   - U prazan vektor dodajte brojeve od 1 do 20 i nakon svakog `push_back` ispišite `size()` i `capacity()`.
   - Ponovite s vektorom za koji je prije petlje pozvan `reserve(100)`, a na kraju pozovite `shrink_to_fit()` i ispišite kapacitet.

2. Korisnik unosi broj dana n, a zatim n izmjerenih temperatura.
   - Vektor temperatura stvorite konstruktorom s n elemenata i popunite ga preko indeksa.
   - Napravite novi vektor promjena temperature između susjednih dana (drugi minus prvi, treći minus drugi, …). Kroz vektor temperatura prođite iteratorima. Za novi vektor unaprijed rezervirajte mjesto.
   - Za izvještaj je potrebno točno 7 dana. Jednim pozivom `resize` skratite vektor na 7 dana ako je uneseno više, odnosno dopunite ga do 7 dana temperaturom zadnjeg izmjerenog dana ako je uneseno manje.

   ```
   Broj dana: 4
   12.5 14 13 9.5
   Promjene: 1.5 -1 -3.5
   Izvjestaj: 12.5 14 13 9.5 9.5 9.5 9.5
   ```

   Koliko elemenata imaju `std::vector<int> a(5);` i `std::vector<int> b{5};`? Provjerite ispisom.

3. Broj dana u tjednu zadajte kao `constexpr int DAYS = 7;`. Sati učenja po danima čuvaju se u `std::array<double, DAYS>`.
   - Napišite funkciju `double total(const std::array<double, DAYS>& hours)` koja vraća ukupan broj sati u tjednu.
   - Napišite funkciju `int best_day(const std::array<double, DAYS>& hours)` koja vraća redni broj dana (od 1) s najviše sati učenja.
   - Za prvi tjedan korisnik unosi sate za svaki dan. Pomoću funkcija ispišite ukupan broj sati i dan s najviše sati učenja.
   - Drugi tjedan popunite s `fill(1.5)`. Usporedite tjedne operatorima `==` i `<`, zamijenite ih sa `swap` i ponovno ispišite ukupne sate.

   Zašto veličina `std::array` ne može biti broj koji korisnik unese? Zašto je `swap` za `std::array` sporiji nego za `std::vector`?

4. Napišite funkciju `bool split_email(const std::string& email, std::string& user, std::string& domain)` koja e-mail adresu dijeli na korisničko ime (dio prije `@`) i domenu (dio iza `@`). Funkcija vraća `false` ako adresa nije ispravna:
   - nema znaka `@` ili ih ima više (`find`, `rfind`),
   - `@` je na početku ili na kraju,
   - u domeni nema točke.

   Adresu korisnik unosi kao cijeli redak (`std::getline`).

   ```
   "ana.horvat@oss.unist.hr" -> korisnik: ana.horvat, domena: oss.unist.hr
   "ana@horvat@oss.unist.hr" -> neispravna adresa
   "ana.horvat@oss"          -> neispravna adresa
   ```

5. Napišite funkcije
   - `std::string with_separators(std::string digits)` koja broju zapisanom kao string umeće točku ispred svake skupine od tri znamenke, gledano s desna (`insert`); broj može biti negativan,
   - `std::string without_separators(std::string text)` koja uklanja sve točke (`find`, `erase`).

```
"1234567" -> "1.234.567"      "-1234567" -> "-1.234.567"
"100000" -> "100.000"         "1.234.567" -> "1234567"
```

Zašto je lakše umetati točke krenuvši od kraja stringa?

6. Napišite funkciju `std::vector<std::string> split_words(const std::string& text)` koja rečenicu dijeli na riječi (`find`, `substr`). Riječi mogu biti odvojene s više razmaka. Za rečenicu koju korisnik unese ispišite:
   - broj riječi i najdulju riječ,
   - riječi obrnutim redoslijedom, spojene znakom `-` u jedan string (`+=`),
   - riječ koja je prva po abecedi (operator `<`).

```
banana   Zebra auto Mrkva  kiwi
broj rijeci: 5
najdulja: banana
kiwi-Mrkva-auto-Zebra-banana
prva po abecedi: Mrkva
```